

MODULE 02

Agent Loop、API 与流式执行

从模型响应到工具反馈：状态机、并发、取消、结果与错误语义

Agent loop 很短，但每个边界都很硬：模型输出必须被结构化解析，工具必须经过权限和生命周期控制，结果必须以可恢复事件重新进入上下文。

研究基线：Claude Code v2.1.236（稳定版） | Python Agent SDK v0.2.141（tag） | 历史源码观察 v2.1.88
资料核验日期：2026-08-19 | 中文深度研究版

目录

- 1. 抽象状态机
- 2. 一轮执行的九步管线
- 3. StreamingToolExecutor 为什么重要
- 4. API 层：模型调用不是简单 HTTP
- 5. 双向 control protocol
- 6. Result 不一定意味着进程可以关闭
- 7. 错误分类与传播
- 8. 停止条件与验证闭环
- 9. 自己复现 loop 的最小骨架

阅读提示：内部模块名以历史源码观察解释，当前功能以官方文档与 v0.2.141 SDK 源码校验。

1. 抽象状态机



官方把工作过程概括为 gather context、take action、verify results。源码视角则更细：每个模型回合都先归一化消息并组装工具池，消费流式 block，遇到 tool_use 后经过权限判定和调度，生成 tool_result，再进入下一次 API 调用；没有工具调用且产生最终文本时结束。 [O2, O24, R2]

不要误读

计划并不是一个独立 planner 服务。大多数时候它只是同一模型在不同 system instructions、tool availability 与 permission mode 下产生的显式文本/计划文件；harness 决定它能否越过执行边界。

2. 一轮执行的九步管线

步骤	输入	动作	输出
1	session state	接收新 prompt/notification	turn origin
2	history + settings	恢复/归一化消息	canonical messages
3	instructions	五类上下文整形与预算检查	API-ready context
4	tools/extensions	组装并序列化 tool schemas	tool catalog
5	API request	流式调用模型	content blocks
6	tool_use	依赖/并发安全分组	execution plan
7	tool call	permission + hooks + sandbox	tool output/error
8	tool_result	截断/持久化/回注	next messages
9	stop condition	继续、等待后台任务或提交 result	ResultMessage

3. StreamingToolExecutor 为什么重要

历史源码分析显示工具调用可以在流中尽早启动，而不是等待整条 assistant message 结束。延迟收益来自把模型继续生成与首个工具准备/执行重叠；代价是取消、半截 JSON、顺序和并发安全复杂度显著上升。 [R1, R2]

- 每个工具声明是否 concurrency-safe；只读搜索通常可并发，文件编辑和依赖前序结果的命令应串行。
- 并发 tool_result 仍需与各自 tool_use_id 对齐，返回顺序不能改变语义。
- 流被中断时必须取消尚未提交的执行，已产生外部副作用的调用不能假装回滚。
- 工具错误不是 Python/JS 异常直接冒泡给模型，而是结构化 tool_result，通常带 is_error=true。

4. API 层：模型调用不是简单 HTTP

请求稳定性取决于 system prompt、tools、beta headers、model、thinking/effort、output schema 和缓存断点。SDK options 最终编译为 CLI 参数或 initialize payload；CLI 再负责 provider/gateway 适配、重试、rate limit、prompt cache 与遥测。 [S3, S5]

维度	控制项	对轨迹的影响
Model	model / fallback / default env	推理能力、成本、工具选择
Thinking	adaptive/enabled/disabled + effort	思考深度与延迟
Budget	max_turns / max_budget_usd / task_budget	停止条件与自我节奏
Schema	json_schema output	结束态验证与结构化结果
Betas	context/task features	协议字段与服务器能力

5. 双向 control protocol

Python SDK 不重写 agent loop，而是把 CLI 当子进程。普通 NDJSON 消息从 stdout 流出；SDK 需要参与的动作走双向控制通道。`Query.initialize()` 把 hooks callback IDs、agents、skill filter、forwardSubagentText 发送给 CLI。 [S2, S3]

方向	subtype	目的
SDK → CLI	initialize	注册 hooks/agents/skills 与能力
CLI → SDK	can_use_tool	请求外部权限决策
CLI → SDK	hook_callback	运行 SDK hook
CLI → SDK	mcp_message	桥接 in-process MCP
SDK → CLI	interrupt/set_model/set_permission_mode	运行时控制
SDK → CLI	rewind_files/mcp_toggle/stop_task	会话与后台任务控制

示例 | 概念化 wire shape；字段以 SDK types 为准

```
{
  "type": "control_request",
  "request_id": "...",
  "request": {
    "subtype": "can_use_tool",
    "tool_name": "Bash",
    "input": {...}
  }
},
{
  "type": "control_response",
  "response": {
    "request_id": "...",
    "subtype": "success",
    "response": {
      "behavior": "allow"
    }
  }
}
```

6. Result 不一定意味着进程可以关闭

v0.2.141 的 `Query._read_messages()` 专门维护 `_inflight_tasks`。一个 turn 的 result 到达时，如果本地 agent/workflow 仍在运行，stdin 必须保持打开，因为后台任务完成后可能触发 follow-up turn，还需要 hook/MCP 控制响应。只有无在途任务的 result 才唤醒关闭输入的 waiter。 [S2]

典型 bug

看到第一个 ResultMessage 就 break，会丢失尾部 system events；看到 result 就关闭 stdin，会让后台 hook 或 SDK MCP 报 Stream closed。官方 Agent SDK 文档也要求把 stream 迭代到完成。 [O24, S2]

7. 错误分类与传播

来源	在消息流中的形态	调用方策略
工具业务错误	tool_result is_error	让模型修正或换方案
预算/最大轮次	result subtype + is_error	读取 subtype，不按字符串猜
API error	error result / status	退避、切换 fallback、保留 session_id
CLI 非零退出	ProcessError	检查先前 error result 与 stderr
协议/JSON 损坏	SDKJSONDecodeError	认为通道失败，不重放副作用
用户取消	Abort/cancel	传播到工具，做有界清理

SDK 会把“CLI 已先发出 error result，随后按设计以 code 1 退出”的场景提升为结构化 `ResultError`，保留 subtype、errors、result、HTTP status、terminal reason 与 raw data，避免只得到自相矛盾的 `success` 或泛化 exit code。 [S2, S4]

8. 停止条件与验证闭环

- 无 tool_use 且有最终文本：候选完成，但结构化输出仍要通过 schema。
- 达到 turn/budget：以终止 subtype 结束，不应伪装 success。
- 有后台 local_agent/local_workflow：等待任务落账与 follow-up result。
- 用户 interrupt：终止当前生成/工具；已经提交的远端副作用需单独核验。
- 验证不是固定最后一步：测试失败会立即成为下一轮 context，形成 feedback loop。 [O2, O24, S2]

9. 自己复现 loop 的最小骨架

示例 | 教学性伪代码：省略了权限、缓存、持久化和错误恢复

```
while True:
    context = shape_context(history, instructions, tools, budget)
    event_stream = model.stream(context)
    assistant, calls = collect_and_maybe_start(event_stream)
    history.append(assistant)
    if not calls:
        return commit_result(history)
    results = await gate_execute_and_observe(calls, abort_signal)
    history.extend(results)
```

如果你的实现只有上面十行，它是 demo，不是 Claude Code 等级的 runtime。生产差距主要来自：确定性上下文预算、协议兼容、并发安全声明、at-most-once 外部动作、取消清理、持久化链和人类审批。

参考资料与证据等级

[O2] 官方 | [How Claude Code works](#)

<https://code.claude.com/docs/en/how-claude-code-works>

[O24] 官方 | [Agent SDK agent loop](#)

<https://code.claude.com/docs/en/agent-sdk/agent-loop>

[S2] 开源源码 | [Internal Query control protocol](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/query.py

[S3] 开源源码 | [Subprocess CLI transport](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/transport/subprocess_cli.py

[S4] 开源源码 | [Message parser](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/message_parser.py

[S5] 开源源码 | [SDK types and options](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/types.py

[R1] 社区研究 | [Dive into Claude Code v2.1.88](#)

<https://github.com/VILA-Lab/Dive-into-Claude-Code>

[R2] 社区研究 | [Claude Code Research - source analysis](#)

<https://github.com/cablate/claude-code-research>

方法说明

官方材料用于确认当前稳定行为；SDK 源码固定在发布 tag v0.2.141；社区材料只用于解释 v2.1.88 历史源码结构。源码行号和内部函数名可能在后续版本变化，外部契约以官方最新文档为准。